Forensic Analysis of TASK.md Subsystem Entropy: Structural Degradation, Log Ephemerality, and Instruction Decay in Agentic Workflows

1. Governance Architecture and the Roots of Entropy

The structural integrity of autonomous software repositories relies heavily on the alignment between governance documentation ("The Constitution") and the executable tools that enforce it. In the context of the Stela repository, this relationship is codified in the **Policy of Truth (PoT.md)**, which establishes a strict hierarchy of canonical surfaces designed to prevent "Drift"—a system failure state defined as any divergence between canon and repository state.[1] However, forensic analysis of the repository's operational subsystems reveals a critical vulnerability at the intersection of static governance and dynamic agent execution: the TASK.md dashboard.

This report investigates the degradation of the TASK.md subsystem, identifying specific mechanisms of entropy arising from the decoupling of maintenance binaries (ops/bin/prune) from the live work surface, the systemic "amnesia" induced by ephemeral logging practices, and the cognitive phenomenon of instruction decay observed in late-stage task execution. By synthesizing internal repository artifacts with broader research on Large Language Model (LLM) behavior, "Lost in the Middle" attention phenomena, and agentic drift, this analysis provides a rigorous, evidence-based roadmap for architectural remediation.

1.1 The Stela Governance Hierarchy

To understand the nature of the observed entropy, one must first deconstruct the intended governance model. The repository operates under a "Filing Doctrine" and "Read-in Order" mandated by PoT.md.[1]

- **PoT.md (The Constitution):** The ultimate authority on staffing, jurisdiction, and physical laws. It explicitly defines "Drift" and "Context Hazard" as failure states requiring immediate cessation of work.[1]
- **SoP.md (The Ledger):** A historical record of the "State of Play." It is an append-only log of what has shipped, providing context on *why* the system exists in its current state but containing no permanent rules.[1]
- **TASK.md (The Contract):** The active work surface. This file is unique in that it serves as both a governance document (containing the Dispatch Packet or DP) and a mutable workspace (containing the Work Log and Session State).[1]

- **docs/MAP.md (The Terrain):** A navigational aid for context continuity.[1]

The theoretical stability of this system relies on the assumption that TASK.md remains synchronized with the rigid expectations of the automation layer (ops/bin/* and tools/lint/*). However, the "Dual State" nature of TASK.md—being both a rigid contract and a fluid scratchpad—creates a friction surface where entropy accumulates faster than the maintenance tools can remediate it.

1.2 The Mechanics of Agentic Entropy

Entropy in AI agent systems is not merely a metaphor for disorder but a measurable divergence in system behavior over time.[2] In the Stela framework, this manifests as "Agent Drift," where the agent's output gradually deviates from the strictures of the Dispatch Packet due to accumulating context noise, evolving models, or ambiguous prompts.[4]

The investigation identifies three primary vectors of entropy within the TASK.md subsystem:

1. **Structural Drift:** The misalignment between the evolving schema of TASK.md (driven by governance updates like DP-OPS-0055) and the static text-processing logic of ops/bin/prune.[1]
2. **Episodic Amnesia:** The systematic deletion of execution context (Work Log) at the end of each task, preventing the accumulation of "negative knowledge" (learning from failure).[5]
3. **Instruction Decay:** The degradation of adherence to strict formatting rules (e.g., Receipt generation) as the task progresses, driven by context window saturation and the "Lost in the Middle" phenomenon.[7]

These vectors do not operate in isolation; they reinforce one another. A structural failure in ops/bin/prune leaves "ghost artifacts" in TASK.md, which increases the noise in the context window, thereby accelerating instruction decay in subsequent sessions. This creates a feedback loop of degradation that threatens the reliability of the entire agentic workflow.

1.3 Methodology of Investigation

This report synthesizes forensic evidence from the repository's own history ledger (SoP.md) and file structure (dump-full-main-2dafc32f.txt) with external research on agent reliability. Key artifacts examined include:

- **DP-OPS-0050:** A documented instance of "Partial Closeout + Scrub Failure," providing direct evidence of the prune binary's inability to handle dirty state.[1]
- **DP-OPS-0051:** A remediation task explicitly aimed at "System Remediation" and "Pointer-First" governance, highlighting the ongoing struggle to stabilize TASK.md.[1]
- **DP-OPS-0058:** The "Tooling Safety and Protocol Hardening" update, which introduced safeguards against uncommitted results, acknowledging the risk of data loss during

pruning.[1]

By tracing the evolution of these DPs, we can reconstruct the failure modes of the TASK.md subsystem and evaluate the efficacy of the current countermeasures.

2. The ops/bin/prune Disconnection: Hardcoding vs. Living Structure

The core structural failure identified in the Stela repository is the disconnection between the ops/bin/prune binary and the TASK.md file it acts upon. TASK.md is designed as a "Living Work Log" and "DP Template" [1], implying a degree of fluidity and evolution. In contrast, ops/bin/prune functions as a rigid hygiene enforcer, relying on hardcoded logic to sanitize the file between sessions.

2.1 The Hardcoded Logic Vulnerability

While the specific source code for ops/bin/prune is not provided in the dump, its behavior can be inferred from the SoP.md operational history and standard patterns of shell-based text processing.[1] Maintenance scripts in such environments typically employ utilities like sed or awk to identify and remove specific sections of a file based on regex anchors (e.g., /^## 5\. Work Log/).

This approach creates a fragility known as "Schema Coupling." The maintenance script is tightly coupled to the specific string representation of the TASK.md structure at a single point in time. When the TASK.md template evolves—as it did in DP-OPS-0055 ("TASK Surface Contract Lock") and DP-OPS-0054 ("TASK Surface Stabilization") [1]—the binary must be updated simultaneously. If it is not, the prune logic fails to match the new headers, resulting in one of two failure modes:

1. **No-Op Failure:** The script fails to find the anchor and does nothing, leaving the old Work Log intact (as seen in DP-OPS-0050).[1]
2. **Destructive Failure:** The script matches a partial pattern and deletes the wrong content, corrupting the new template.

The "disconnection" cited in the user query is effectively a synchronization lag. The TASK.md structure is "live" and subject to frequent governance refinements, while ops/bin/prune is a compiled or scripted artifact that lags behind, creating a "Drift Gap."

2.2 Forensic Evidence: The DP-OPS-0050 Scrub Failure

The entry for **DP-OPS-0050** in SoP.md provides smoking gun evidence of this disconnection. The ledger records: "deviations and anomalies: session began from a dirty tree... ./ops/bin/prune --dp=DP-OPS-0050 --scrub was intentionally not run; TASK closeout state

remained partial and required DP-OPS-0051 remediation".[1]

This incident demonstrates that the prune binary is not robust enough to handle deviations. When the system entered a "dirty state" (a violation of PoT.md Section 2), the operator was forced to bypass the pruning tool entirely, likely because the tool's hardcoded logic could not account for the anomalous state of the TASK.md file. This forced a manual intervention in DP-OPS-0051 to "repair ledger continuity" and "eliminate TASK.md ghost-canon drift vectors".[1] The need for a subsequent DP solely to fix the mess left by the previous DP indicates a fundamental failure in the automated hygiene subsystem.

2.3 The "Reset-Task" Anomaly

The introduction of the --reset-task flag in ops/bin/prune (documented in DP-OPS-0058 [1]) represents an attempt to mitigate this fragility. The explicit purpose of this flag is to "hardened ops/bin/prune with an uncommitted RESULTS guard and explicit --reset-task, decoupled --scrub from TASK reset behavior."

This decoupling acknowledges that "scrubbing" (cleaning a file in place) and "resetting" (restoring a file to baseline) are distinct operations with different risk profiles. Scrubbing relies on parsing the *current* (potentially corrupted) file, while resetting *should* rely on a pristine source. However, if --reset-task still relies on hardcoded string injection rather than copying a distinct template file, it remains vulnerable to the same entropy. If the hardcoded template inside the binary differs from the governance-mandated structure in docs/ops/specs/surfaces/task.md [1], the "reset" operation effectively enforces an obsolete state, creating instant drift upon execution.

2.4 Structural Drift and Regular Expression Fragility

The reliance on pattern matching for structural enforcement is inherently flawed in agentic systems. Agents are prone to "free-styling" or minor formatting deviations—adding an extra newline, changing a header level, or appending clarifying text to a title.[9]

For example, if ops/bin/prune expects:

5. Work Log

But the agent writes:

5. Work Log (2026-02-13)

A strict regex anchor (^## 5\. Work Log$) will fail to identify the section. The script will assume the section is missing or, worse, identify it as part of the previous section, leading to a failure to truncate the log. This accumulates "ghost text" at the bottom of the file—vestigial logs from previous sessions that pollute the context window of future agents, contributing to

"Context Rot".[11]

2.5 Remediation: The Golden Master Pattern

To resolve the disconnection, the dependency arrow must be reversed. ops/bin/prune should not contain the logic to *parse* TASK.md; it should only contain the logic to *replace* it. The system requires a **Golden Master Template** (.github/templates/TASK.tpl.md) that serves as the single source of truth for the blank state.

Instead of:

> Prune: Read TASK.md -> Find Regex -> Delete Lines -> Save TASK.md

The logic must be:

> Prune: Read Template -> Inject Session Pointers -> Overwrite TASK.md

This "Stateless TASK" architecture ensures that regardless of how badly an agent corrupts the structure during a session, the closeout process always restores a mathematically perfect baseline, eliminating structural entropy.[6]

3. The Ephemerality of the Work Log: Systemic Amnesia

Section 5 of TASK.md, the "Work Log," is designed as a transient scratchpad. The TASK.md template explicitly initializes this section as "(No active thread)".[1] Upon the completion of a Dispatch Packet, the standard procedure involves "resetting" the Work Log, effectively deleting the chronological record of the agent's reasoning and execution steps. This design choice, while maintaining a clean immediate workspace, introduces a severe long-term cognitive deficit known as "Systemic Amnesia."

3.1 The Loss of "Negative Knowledge"

In AI systems, "Negative Knowledge"—information about what *didn't* work, which paths were dead ends, and which tools failed—is as valuable as positive knowledge (what worked).[5] The SoP.md ledger captures the *outcome* ("Refactored navigation module"), but the *process* (the trial and error, the dependency conflicts encountered, the logic for choosing one library over another) is contained solely in the Work Log.

When ops/bin/prune wipes Section 5, this negative knowledge is destroyed. A subsequent agent tasked with a related objective has no way of knowing that a specific approach was already attempted and failed. This forces the agent to re-derive the same failure modes, wasting tokens and compute resources in a redundant discovery cycle.[14] This is analogous to a software team deleting all Jira comments and Pull Request discussions after merging a

feature, leaving only the commit message.

3.2 The Limitations of SoP.md as Memory

The repository relies on SoP.md as the "History Ledger".[1] However, the SoP.md entries are high-level summaries. For example, the entry for **DP-OPS-0056** reads:

> "Objective: Hardened Task subsystem governance by making tools/lint/task.sh the sole TASK container enforcer... Verification: bash tools/lint/context.sh..." [1]

This summary lacks the granular "Episodic Memory" of the session. It does not record *why* the previous linter configuration was failing, or the specific edge cases that necessitated the change. This forces agents to operate with "Semantic Memory" (general facts) but without "Episodic Memory" (specific experiences), limiting their ability to learn from the repository's operational history.[5]

3.3 Context Rot and Ghost Logs

While the intent is to clear the Work Log, the forensic analysis of "ghost canon drift vectors" (referenced in DP-OPS-0051 [1]) suggests that this clearing is often imperfect. If prune fails to perfectly identify the start and end of the log (due to the regex fragility discussed in Section 2.1), fragments of old logs may persist.

These fragments occupy the "tail" of the context window. As discussed in research on "Context Dilution," unrelated text at the end of a prompt can disproportionately degrade the model's ability to follow instructions.[15] A TASK.md file cluttered with 50 lines of leftover logs from three DPs ago introduces noise that distracts the agent from the current Freshness Gate or Preflight instructions.

3.4 Persistent Persistence Patterns

To arrest this entropy, the Work Log must be transitioned from a transient section to a persistent artifact. Current research in agentic persistence suggests a "Handoff Protocol" where the session state is serialized into an immutable record before the workspace is cleared.[6]

Instead of deleting Section 5, the system should:

1. **Serialize:** Move the content of Section 5 to storage/handoff/LOG-<DP-ID>-<TIMESTAMP>.md.
2. **Link:** Update the SoP.md entry to include a pointer to this log file.
3. **Index:** Allow future agents to retrieve these logs via semantic search or direct reference when encountering similar tasks.[17]

This converts the Work Log from "Trash" (under the Filing Doctrine) to "Archive," preserving

the rich context of decision-making without cluttering the active interface.

4. Instruction Decay: The Section 3.4.5 Threshold

The user query highlights a specific behavioral trend: "the tendency for writers to deviate after Section 3.4.5." Section 3.4.5 of TASK.md is the "Receipt" section, requiring the execution of verification commands and the pasting of specific outputs (OPEN path, DUMP path, diff stats).[1] This section invariably appears at the end of the Dispatch Packet execution flow.

4.1 The "Lost in the Middle" Phenomenon

Research into LLM performance consistently identifies a "U-shaped" attention curve, where models perform best at the beginning and end of a context window, but suffer significant degradation in the middle.[8] This is often termed the "Lost in the Middle" phenomenon.

In the context of TASK.md, the "Receipt" section (3.4.5) is structurally located at the end of the template. However, during execution, the agent fills in Sections 3.4.1 (State), 3.4.2 (Request), 3.4.3 (Changelog), and 3.4.4 (Patch). By the time the agent reaches 3.4.5, the "middle" of the context window is saturated with the agent's own generated reasoning, code snippets, and diffs. The strict instructions defining the *format* of the receipt—originally loaded at the start of the session—are now "lost in the middle," buried under thousands of tokens of execution trace.[19]

4.2 Cognitive Load and Mode Switching

The transition to Section 3.4.5 represents a significant "Mode Switch" for the agent.

- **Phases 3.4.1 - 3.4.4:** Creative/Architectural Mode. The agent is generating code, solving problems, and reasoning abstractly. This requires a higher "temperature" or flexibility.
- **Phase 3.4.5:** Auditor/Clerk Mode. The agent must mechanically run scripts (tools/lint/*.sh) and paste outputs exactly, without creativity or summarization.

LLMs struggle to switch from a high-entropy creative mode to a low-entropy compliance mode within a single generation stream.[9] The momentum of the "creative" generation bleeds into the "clerical" phase, leading to:

- **Hallucination of Checks:** Marking lints as "PASS" without running them.
- **Formatting Slop:** Summarizing the git diff instead of pasting the raw output.
- **Skipping Mandatory Blocks:** Omitting the "Mandatory Closing Block" required for the PR description.[1]

4.3 Attention Decay Mechanics

The "Tail-End Attention Drop" is exacerbated by the length of the prompt. As the context grows, the model's ability to attend to specific formatting constraints defined early in the

prompt diminishes.[21] In TASK.md, the instructions for the Receipt are often defined in the template *before* the execution plan. By the time the agent *executes* the plan, those instructions are distant memories in the token stream.

4.4 Remediation: Tail-End Injection (The Recency Effect)

To counteract instruction decay, the system must leverage the "Recency Effect".[21] Critical instructions regarding the Receipt and Closeout must be re-injected into the context *immediately before* the agent attempts Section 3.4.5.

This requires a "Just-In-Time" (JIT) prompt injection strategy. The TASK.md template should include a forceful "Stop and Read" anchor at the end of Section 3.4.4:

> "**STOP.** You have completed the execution plan. The following section (3.4.5) requires **STRICT ADHERENCE** to verification protocols. Do not invent outputs. Run the commands literally. Paste the raw output."

This refreshes the agent's attention on the specific constraints of the Receipt phase, overriding the "creative" inertia from the previous sections.

5. Evaluation of Existing Lints and CI Gates

The Stela repository employs a suite of automated enforcers to maintain order. Evaluating these tools reveals a mix of effective static analysis and critical semantic blind spots.

5.1 tools/lint/dp.sh (The DP Linter)

This script is tasked with validating the Dispatch Packet schema (Section 3).

- **Function:** It scans TASK.md for the presence of required headers (### 3.1 Freshness Gate) and fields.[1]
- **Efficacy:** It effectively prevents "Gross Structural Deletion." An agent cannot accidentally delete the Freshness Gate without this linter failing the PR.
- **Blind Spot:** It performs *syntactic* rather than *semantic* validation. It checks that the "Receipt" header exists, but it cannot easily verify if the *content* of the receipt is truthful. It cannot detect if the pasted git diff actually matches the file system, or if the "Verification" output was hallucinated.[23] It validates the *container*, not the *payload*.

5.2 tools/lint/task.sh (The Container Linter)

This script enforces the overall TASK.md structure (Sections 1-5).

- **Function:** Ensures the Session State is pointer-first and that the Work Log anchor exists.
- **Fragility:** As discussed in Section 2, this linter is tightly coupled to the template. If the governance evolves (e.g., adding a "Safety" section), this linter becomes a blocker until it is updated. This creates "Maintenance Friction," where every governance change requires

a synchronized code change in the tools.[24]

5.3 CI Gates (repo_gates.yml)

The repo_gates workflow runs these lints on every Pull Request.[1]

- **Status:** It effectively blocks "dirty" commits from reaching main.
- **Deficiency:** The CI gate sees the TASK.md file as a static artifact. It does not understand the *temporal* execution of the task. It cannot verify that the steps in the Work Log were actually performed in the order stated. It certifies the *form* of the submission, not the *rigor* of the process.

6. Feasibility and Risk of Stricter Worker-Side Policing

The user query investigates the feasibility of "stricter worker-side policing before work commencement." This refers to "Pre-Flight Gates" that block the agent from generating code until the administrative state (DP plan) is valid.

6.1 Feasibility of Pre-Flight Enforcement

Implementing strict pre-flight checks is highly feasible using existing hooks.

- **Mechanism:** A wrapper script (ops/bin/agent-wrapper or similar) can run tools/lint/dp.sh *immediately after* the agent generates the DP plan but *before* it is allowed to edit any project files.[25]
- **Precedent:** The "DP Preflight Gate" (Section 3.1.1) already exists in the template.[1] The proposal is to move this from a "suggested step" to a "system-enforced hard gate."

6.2 The Risk of False Positives and "Deadlock"

The primary risk of strict pre-flight policing is **"False Positive Paralysis"** or **"Guardrail Recursion Deadlock"**.[26]

Scenario: An agent is tasked with refactoring the lint/dp.sh script itself because the linter contains a bug or an obsolete rule.

1. **Action:** The agent writes a plan to fix lint/dp.sh.
2. **Enforcement:** The system runs the *existing* (buggy) lint/dp.sh against the plan.
3. **Failure:** The buggy linter flags the valid plan as a violation.
4. **Deadlock:** The agent is blocked from fixing the tool because the tool prevents the fix.

This "Self-Modification Paradox" is a critical risk in agentic systems. If the policing is too strict, the system becomes rigid and incapable of self-repair or evolution.[27]

6.3 Alert Fatigue

Furthermore, if the policing is overly pedantic (e.g., failing a task due to a missing whitespace in a log entry), the agent (and the human operator) suffers from "Alert Fatigue".[28] Critical warnings about Context Hazards get ignored because they are buried in a sea of trivial formatting complaints. Policing must distinguish between "Safety Critical" violations (e.g., touching forbidden files) and "Style" violations.

7. Findings and Solutions

Based on this forensic investigation, the following Findings and Solutions are presented to remediate the entropy in the TASK.md subsystem.

Findings

#	Finding	Root Cause	Implication
F1	**Prune/TASK Disconnect**	ops/bin/prune uses hardcoded regex logic that drifts from the evolving TASK.md template.	Incomplete scrubbing, "ghost" text, and corruption of the dispatch surface.
F2	**Work Log Amnesia**	Section 5 is explicitly designed to be ephemeral and is wiped at closeout.	Loss of "negative knowledge" and episodic memory; agents repeat past mistakes.
F3	**Receipt Deviation**	Agent performance degrades at Section 3.4.5 due to "Lost in the Middle" / Context Saturation.	Invalid receipts, hallucinated verification, and failure to run mandatory lints.
F4	**Linter Fragility**	dp.sh and task.sh validate syntax, not semantics, and are coupled to static schemas.	Governance changes break tooling; lints pass despite semantic hallucinations.
F5	**Policing Deadlock**	Strict pre-flight	Agents cannot fix

		gates lack "Bootstrap Mode" for self-modification tasks.	broken tools because the broken tools block the fix.

Solutions

Solution 1: The Golden Master Pattern (Decoupling Prune)

Objective: Eliminate the fragility of hardcoded regex in ops/bin/prune.

Mechanism:

1. Create .github/templates/TASK.tpl.md as the immutable Source of Truth for the blank task structure.
2. Refactor ops/bin/prune to **replace** TASK.md rather than **scrub** it.
3. **Logic:**
   - Read .github/templates/TASK.tpl.md.
   - Inject the current Session State pointers (from ops/bin/open output).
   - Overwrite TASK.md with this pristine artifact.
     Benefit: Changes to the TASK structure only need to be made in the template file. The binary becomes agnostic to the content, preventing structural drift.

Solution 2: Persistent Handoff Logging (Solving Amnesia)

Objective: Preserve episodic memory without cluttering the active workspace.

Mechanism:

1. Deprecate the "reset" (deletion) of the Work Log.
2. Implement an **Archive-and-Link** strategy in ops/bin/prune:
   - Extract the content of Section 5.
   - Write it to storage/handoff/LOG-<DP-ID>-<TIMESTAMP>.md.[6]
   - Append a link to this log in the corresponding SoP.md entry.
     Benefit: Creates a permanent, searchable audit trail of *how* work was done, enabling future agents to learn from past execution traces.

Solution 3: Tail-End Compliance Injection (Fixing Deviation)

Objective: Counteract "Lost in the Middle" attention decay at Section 3.4.5.

Mechanism:

1. Modify ops/bin/open or the agent system prompt to include a **Compliance Anchor** at the very end of the context window.
2. **Anchor Text:** "CRITICAL: You are approaching the Receipt phase (3.4.5). You must

execute tools/verify.sh and paste the output exactly. Failure to produce a valid receipt is a system failure. Do not summarize." **Benefit:** Leverages the Recency Effect to refresh the agent's adherence to formatting rules immediately before the critical verification step.[21]

Solution 4: The "Dry Run" Pre-Flight with Override

Objective: Strict policing without deadlock.

Mechanism:

1. Implement ops/bin/dry-run which executes tools/lint/dp.sh against the *proposed* DP plan.
2. Integrate this into the workflow: Agents must pass dry-run before executing code.
3. **Safety Valve:** Include an --admin-override flag in dry-run that requires an explicit "reason" field. This allows agents to modify the linters themselves without entering a recursion deadlock, provided they justify the bypass.[25]

Solution 5: Semantic Linter Upgrades

Objective: Move beyond regex checks.

Mechanism:

1. Upgrade tools/lint/dp.sh to include **Existence Checks**:
   - Verify that files listed in the "Allowlist" actually exist.
   - Verify that the "Base HEAD" hash in Section 3.1 matches the actual git HEAD.
2. Implement **Receipt Validation**: Parse the pasted git diff --stat in Section 3.4.5 and compare it against the actual file system state. If they mismatch, fail the lint (catching hallucinated receipts).

8. Conclusion

The entropy observed in the TASK.md subsystem is a predictable consequence of managing dynamic agentic workflows with static, regex-based tooling. The current architecture places an unsustainable cognitive load on the agent to maintain structure while simultaneously performing creative work, leading to inevitable degradation as context windows fill.

By shifting from a paradigm of "Hygiene via Scrubbing" to "Hygiene via Regeneration" (Golden Master), and by converting ephemeral logs into persistent assets, the repository can arrest this drift. Furthermore, recognizing the cognitive limits of LLMs through "Tail-End Injection" and implementing "Dry Run" gates with safety valves provides a robust framework for scaling agent operations without succumbing to chaos. The transition is from a system that *hopes* for compliance to one that *architects* for it.

Works cited

1. dump-full-main-2dafc32f.txt
2. Agentic Foundation: Managing Entropy in AI Systems | by Oleg Komissarov -

Medium, accessed February 13, 2026, https://medium.com/@olegkomissarov/agentic-foundation-managing-entropy-in-ai-systems-e5833e76c768
3. AI Agent Entropy - Why More Isn't Always Better - RL Canning, accessed February 13, 2026, https://www.rlcanning.com/insights/ai-agent-entropy-why-more-isnt-always-better/
4. Managing AI Agent Drift: How to Maintain Consistent Performance Over Time - Maxim AI, accessed February 13, 2026, https://www.getmaxim.ai/articles/managing-ai-agent-drift-how-to-maintain-consistent-performance-over-time/
5. Building smarter AI agents: AgentCore long-term memory deep dive - AWS, accessed February 13, 2026, https://aws.amazon.com/blogs/machine-learning/building-smarter-ai-agents-agentcore-long-term-memory-deep-dive/
6. Persistence Patterns for AI Agents That Survive Restarts - DEV Community, accessed February 13, 2026, https://dev.to/aureus_c_b3ba7f87cc34d74d49/persistence-patterns-for-ai-agents-that-survive-restarts-59ck
7. Context rot explained (& how to prevent it) - Redis, accessed February 13, 2026, https://redis.io/blog/context-rot/
8. Lost in the Middle: How Language Models Use Long Contexts, accessed February 13, 2026, https://teapot123.github.io/files/CSE_5610_Fall25/Lecture_12_Long_Context.pdf
9. a trick that makes LLMs follow instructions way more tightly : r/PromptEngineering - Reddit, accessed February 13, 2026, https://www.reddit.com/r/PromptEngineering/comments/1p4ksjp/a_trick_that_makes_llms_follow_instructions_way/
10. does anyone have a reliable trick for keeping llms consistent across long workflows? : r/PromptEngineering - Reddit, accessed February 13, 2026, https://www.reddit.com/r/PromptEngineering/comments/1p2zhde/does_anyone_have_a_reliable_trick_for_keeping/
11. Context Rot: Why AI Gets Worse the Longer You Chat (And How to Fix It) - Product Talk, accessed February 13, 2026, https://www.producttalk.org/context-rot/
12. Mixed Markdown and Shell Scripting - GitHub Gist, accessed February 13, 2026, https://gist.github.com/pjeby/c137ace4d91e61e8f1f80e92d84e8b70
13. AI Agent Memory: Build Stateful AI Systems That Remember - Redis, accessed February 13, 2026, https://redis.io/blog/ai-agent-memory-stateful-systems/
14. 2 years building agent memory systems, ended up just using Git : r/AI_Agents - Reddit, accessed February 13, 2026, https://www.reddit.com/r/AI_Agents/comments/1mw4jvp/2_years_building_agent_memory_systems_ended_up/
15. Context Dilution: When More Tokens Hurt AI - diffray, accessed February 13, 2026, https://diffray.ai/blog/context-dilution/

16. The Physics of Prompting: Why Words Don't Matter, But Structure Does - Towards AI, accessed February 13, 2026, https://pub.towardsai.net/the-physics-of-prompting-why-words-dont-matter-but-structure-does-4c26cd69fc06
17. Memory for AI Agents: Designing Persistent, Adaptive Memory Systems | by Nimeth Nimdinu, accessed February 13, 2026, https://medium.com/@20011002nimeth/memory-for-ai-agents-designing-persistent-adaptive-memory-systems-0fb3d25adab2
18. Lost in the Middle in LLMS. Why large language models ignore the... | by Cengizhan Bayram | Medium, accessed February 13, 2026, https://medium.com/@cenghanbayram35/lost-in-the-middle-in-llms-86e461dc7212
19. Found in the Middle: How Language Models Use Long Contexts Better via Plug-and-Play Positional Encoding - arXiv, accessed February 13, 2026, https://arxiv.org/html/2403.04797v1
20. One year of agentic AI: Six lessons from the people doing the work - McKinsey, accessed February 13, 2026, https://www.mckinsey.com/capabilities/quantumblack/our-insights/one-year-of-agentic-ai-six-lessons-from-the-people-doing-the-work
21. Prompt Engineering: Stable output format in long contexts | by Lilian Li | Feb, 2026 | Medium, accessed February 13, 2026, https://medium.com/@lilianli1922/prompt-engineering-stable-output-format-in-long-contexts-530126396863
22. Next Gen LLM Prompting. A Guide for Practical Results | by Julian B | Medium, accessed February 13, 2026, https://medium.com/@julian.burns50/next-gen-llm-prompting-7b92f10f1855
23. How To Validate AI Agents in Business Systems - AI Tools - God of Prompt, accessed February 13, 2026, https://www.godofprompt.ai/blog/validate-ai-agents-business-systems
24. Evaluating Large Language Models at Evaluating Instruction Following - OpenReview, accessed February 13, 2026, https://openreview.net/forum?id=tr0KidwPLc
25. claude-code-collective-newworldasia-v3 - Yarn Classic, accessed February 13, 2026, https://classic.yarnpkg.com/en/package/claude-code-collective-newworldasia-v3
26. Linter rules - Dart, accessed February 13, 2026, https://dart.dev/tools/linter-rules
27. The Dangers of Unregulated AI in Policing | Brennan Center for Justice, accessed February 13, 2026, https://www.brennancenter.org/our-work/research-reports/dangers-unregulated-ai-policing
28. Prioritizing Real-Time Failure Detection in AI Agents - Partnership on AI, accessed February 13, 2026, https://partnershiponai.org/wp-content/uploads/2025/09/agents-real-time-failure-detection.pdf?vgo_ee=zBAC1la9zQyJHSnpG6BgMHYqtA2DVnJlxaZdlyzMse4LqANZiVSdqdBDKQ%3D%3D%3AUuOdAvb8Al76ab6ZrhxDyj0LJ66FZeBh